

HYPERGC: Learning Hypergraph Representations via Full Hyperedge Reconstruction and Contrastive Evaluation

David Yoon Suk Kang*
dyskang@cbnu.ac.kr
Chungbuk National University
South Korea

So-Bin Jung*
sobin98@hanyang.ac.kr
Hanyang University
South Korea

Sang-Wook Kim†
wook@hanyang.ac.kr
Hanyang University
South Korea

Abstract

Hypergraph representation learning (HRL) is essential for modeling complex groupwise relationships in real-world data. However, existing generative self-supervised HRL methods suffer from two key limitations: (C1) partial hyperedge reconstruction fails to capture high-order formation semantics, and (C2) negative-sampling-based evaluation is unstable and sensitive to sample quality. To address these issues, we propose HYPERGC, a hybrid self-supervised HRL framework that unifies generative and contrastive learning through two core strategies: (S1) full hyperedge reconstruction via progressive selection with dynamic target cross-entropy, and (S2) multi-view contrastive evaluation without handcrafted negative samples. Experiments on 11 real-world hypergraph datasets across node classification, hyperedge prediction, and community detection show that each strategy is effective individually, while their combination consistently shows the best performance, outperforming 19 state-of-the-art HRL methods by up to 12.27%, 5.12%, and 34.35% on the respective tasks.

CCS Concepts

• Computing methodologies → Machine learning.

Keywords

Hypergraph representation learning, self-supervised learning, generative learning, contrastive learning, progressive-selection

ACM Reference Format:

David Yoon Suk Kang, So-Bin Jung, and Sang-Wook Kim. 2026. HYPERGC: Learning Hypergraph Representations via Full Hyperedge Reconstruction and Contrastive Evaluation. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770855.3817691>

1 Introduction

A *hypergraph* generalizes a graph by allowing a hyperedge to connect more than two nodes, unlike a conventional edge that connects only a pair of nodes [7, 20, 48]. Hypergraphs have attracted growing attention for modeling complex structural information across

diverse domains, ranging from social networks to molecular systems [5, 13, 18, 33, 34, 44, 46]. Accordingly, they have been widely applied to tasks such as recommendation [13, 44], node classification [8, 10, 42], community detection [5, 33, 34], and hyperedge prediction [18, 46].

To support these tasks, *hypergraph representation learning* (HRL) maps nodes into vectors a low-dimensional space in such a way to preserve hypergraph structural properties [3, 10, 20, 22, 25, 48]. Existing HRL methods largely fall into two paradigms. *Semi-supervised HRL* leverages limited labeled nodes to learn task-specific vectors and achieves strong performance on node classification and hyperedge prediction [8, 10, 42], but degrades under sparse or noisy labels often generalizing poorly beyond the training task [22, 25]. In contrast, *self-supervised HRL* derives supervision directly from hypergraph structure [9, 22, 25, 39], enabling richer structural representation learning. Self-supervised HRL is typically categorized into contrastive and generative approaches [9, 22, 25, 39]; in this work, we focus on a generative learning paradigm due to its stronger capacity to model complex groupwise relationships [9, 22].

An essential goal of generative HRL is to generate hyperedges that faithfully reflect how they are formed in real-world hypergraphs; it thus obtains the final node vectors after the training is finished. Our preliminary analysis on real-world hypergraphs with temporal snapshots shows that hyperedges do not emerge abruptly in a fully formed manner; instead, they evolve gradually through *incremental node additions* (see Figure 1 in Section 3). However, existing generative methods typically rely on *partial reconstruction* [9, 22], where a subset of nodes in a hyperedge is masked and reconstructed from the remaining ones. While this strategy can improve training stability, it introduces a key challenge (C1): limited ability to capture the complex groupwise relationships inherent in real-world hyperedge formation [11, 40].

Another critical component is evaluating whether generated hyperedges preserve the structural properties of the original hypergraph [9, 22]. Existing methods commonly adopt a *discrimination-based evaluation paradigm*, treating real hyperedges as positives and randomly constructed node sets as negatives. To this end, a binary classifier is trained accordingly, and the quality of generated hyperedges is measured by their likelihood of being classified as positives. However, this paradigm poses another challenge (C2): strong dependence on the hardness and quality of negative samples, which can substantially affect training effectiveness and robustness [11, 40].

To address these challenges, we propose HYPERGC, a hybrid self-supervised HRL framework built on a generative learning foundation and augmented with a contrastive objective, comprising two key strategies: (S1) *full reconstruction-based hyperedge generation*

*Two first authors have contributed equally to this work.

†Corresponding author.



and (S2) *contrastive learning-based evaluation*. In (S1), HYPERGC reconstructs hyperedges via *progressive selection*, starting from a seed node and iteratively adding a contextually-relevant node until the size of a target hyperedge is reached, thereby reflecting real-world hyperedge formation.

A key challenge in (S1) is accurate next-node selection. To address this, HYPERGC leverages an LSTM [15] to model sequential dependencies among previously selected nodes, together with an attention mechanism [35] to score candidate nodes by relevance. In addition, since the supervision signal changes dynamically during node-by-node reconstruction, we propose a *dynamic target cross-entropy (DTCE) loss*, where the target set is updated at each step by removing a correctly predicted one, and the loss is recomputed with respect to the updated target set.

In (S2), HYPERGC adopts a contrastive learning-based evaluation to assess the quality of generated hyperedges without relying on negative sampling. Specifically, HYPERGC first constructs two augmented views of the hypergraph and contrasts generated hyperedges in one view with their original counterparts in the other. By contrasting matched hyperedges across the two augmented views against unmatched ones, the model enforces relational consistency and refines node vectors progressively throughout training.

To validate HYPERGC, we conduct extensive experiments with 19 state-of-the-art HRL methods on 11 real-world hypergraph datasets across three downstream tasks: node classification, hyperedge prediction, and community detection. The results show that each proposed strategy is effective on its own, while their combination consistently yields the best performance. Overall, HYPERGC outperforms significantly 19 state-of-the-art HRL methods, achieving gains of up to 12.27%, 5.12%, and 34.35% on the respective three tasks above.

The contributions of this paper are summarized as follows:

- **Important observation.** Through the analysis of 4 real-world hypergraph datasets with timestamps, we observe that individual hyperedges tend to grow gradually by adding nodes over time, rather than emerging fully formed at once.
- **Challenges.** We revisit existing generative HRL methods and highlight two challenges in their design.
 - (C1) Partial reconstruction in hypergraph generation
 - (C2) Discrimination-based evaluation of generated hyperedges
- **Novel framework.** To address these challenges, we propose HYPERGC, a new hybrid self-supervised HRL framework that integrates two strategies:
 - (S1) *Progressive selection with the DTCE loss* for full hyperedge reconstruction, addressing challenge (C1).
 - (S2) *Contrastive learning-based evaluation* that eliminates the need for explicitly generated negative samples, addressing challenge (C2).
- **Extensive evaluation.** We conduct extensive experiments in comparison with 19 state-of-the-art HRL methods on 11 real-world hypergraph datasets across three downstream tasks.

2 Related Work

Semi-supervised HRL. Semi-supervised HRL leverages a small set of labeled nodes alongside a large number of unlabeled ones during training [3, 8, 10, 17, 36, 38, 42]. Most methods build on Graph

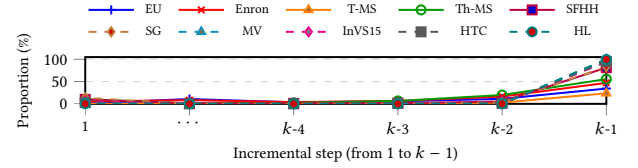


Figure 1: Incremental node additions for hyperedge formation in real-world hypergraphs.

Neural Networks (GNNs) [23, 35], typically transforming hypergraph structures into graph-like representations to enable message passing. HGNN [10] and HyperGCN [42] convert each hyperedge into a *clique*, enabling direct GNN application. HNHN [8] models the hypergraph as a *bipartite graph* between nodes and hyperedges. UniGNN [17] introduces a unified message-passing scheme that generalizes to both graphs and hypergraphs. AllSet [3] integrates GNNs with permutation-invariant set encoders to represent hyperedges as sets and capture groupwise relationships. ED-HNN [36] introduces permutation-equivariant diffusion operators to encode hypergraph structures. PhenomNN [38] learns a hypergraph via a regularization-based energy function guided message passing.

Self-supervised HRL. Self-supervised HRL constructs supervision signals directly from the hypergraph topology *without relying on labels* [9, 22, 25, 39]. This paradigm generally offers stronger generalization than semi-supervised ones by capturing rich relational information in an unsupervised manner. Self-supervised HRL can be broadly divided into two paradigms: contrastive and generative learning. *Contrastive learning* aims to model similarity and dissimilarity among nodes and hyperedges [25, 39]. TriCL [25] introduces a tri-directional contrastive loss to jointly model node, hyperedge, and membership semantics. HyperGCL [39] generates multiple augmented hypergraph views and applies contrastive learning across them. SE-HSSL [27] improves sampling efficiency by constructing self-supervised signals from informative structural relationships, reducing the reliance on extensive negative sampling. VilLain [26] enhances self-supervised HRL through virtual label propagation guided by higher-order label homogeneity. *Generative learning*, on the other hand, focuses on reconstructing or generating hyperedges from the observed structure [9, 22]. HyperGRL [9] and HypeBoy [22] adopt a *partial reconstruction* strategy, where some nodes in a hyperedge are masked and the model predicts the masked nodes from the remaining visible ones.

3 Hyperedge Formation in Real-World

The core objective of generative HRL is to reconstruct real-world hypergraphs. While existing generative HRL methods [9, 22] focus only on accurately generating hyperedges, we claim that the hyperedge generation needs to follow the *process* by which real-world hyperedges are formed for effective HRL. To validate this claim, we examine hyperedge formation mechanisms in real-world hypergraphs.

Prior studies show that real-world group relationships are formed through three primary mechanisms: expansion of existing groups, contraction, and the formation of new groups [2]. Accordingly, we analyze real-world hypergraphs to quantify the prevalence of these mechanisms in practice. For this analysis, we focus on those hyperedges whose sizes are greater than the average hyperedge size, as

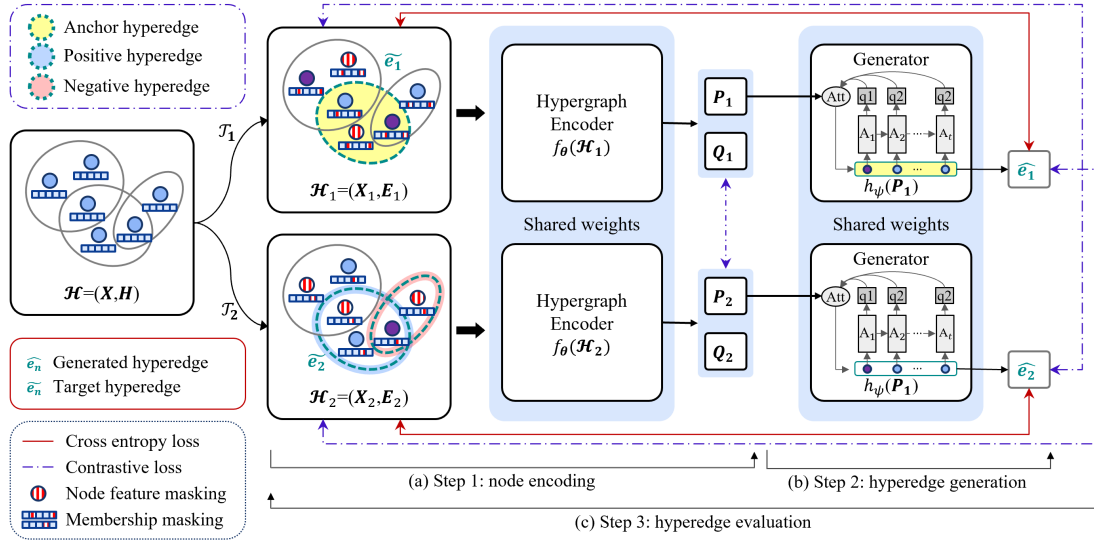


Figure 2: Overview of representation learning in HYPERGC.

larger hyperedges more reliably capture meaningful group formation dynamics. We conduct this study on ten real-world hypergraph datasets with temporal information. These datasets span email communication, Web-based Q&A platforms [1], and human contact networks collected from diverse real-world domains, including conferences, exhibitions, communities, offices, and hospitals [24]. Our analysis shows that, across all datasets, most hyperedges (99.15%) are formed through progressive growth, while contraction and new hyperedge formation occur only infrequently (see Appendix A.1).

Then, we examine the number of node-addition steps required for each hyperedge to get formed. Figure 1 presents the results. The x -axis represents the number of incremental steps needed to complete a hyperedge of size k . In other words, the value of $(k - i)$ indicates hyperedges of size k are ultimately formed after exactly $(k - i)$ node addition steps. The y -axis represents the proportion of such hyperedges within a corresponding dataset. We observe that, in most datasets, hyperedges of size k are typically formed in an incremental way through $(k - 2)$ or $(k - 1)$ node additions before reaching their final size. This indicates that real-world hyperedges tend to *evolve incrementally* rather than abruptly appearing fully-formed, suggesting that generative HRL methods should capture such progressive construction. If generative HRL can incorporate such a hyperedge formation process into its training, it is expected to achieve better performance than existing methods. Motivated by this observation, we focus on developing a generative HRL approach capable of modeling and leveraging the way of real-world hyperedge generation.

4 HYPERGC: The Proposed Framework

4.1 Notations and Overview

A hypergraph is defined as $\mathcal{H} = (V, E)$, where V denotes the set of nodes and E the set of hyperedges. Its structure is captured by an incidence matrix $\mathbf{H} \in \{0, 1\}^{|V| \times |E|}$, where $\mathbf{H}(v, e) = 1$ indicates that node v is incident to hyperedge e . Each hyperedge is associated with a positive weight, represented by a diagonal matrix $\mathbf{W} \in \mathbb{R}^{|E| \times |E|}$.

Node features are given by $\mathbf{X} \in \mathbb{R}^{|V| \times F}$, and we equivalently represent the hypergraph as $\mathcal{H} = (\mathbf{X}, \mathbf{H})$. The node and hyperedge degree matrices are denoted by $\mathbf{D}_V$ and $\mathbf{D}_E$, respectively. In this paper, we address the problem of hypergraph representation learning, which aims to learn a hypergraph encoder $f_\theta: V \rightarrow \mathbb{R}^{|V| \times F'}$ that maps each node $v_i \in V$ to an F' -dimensional vector $\mathbf{p}_i \in \mathbb{R}^{F'}$. The objective is to make the node vectors to preserve both the structural properties of the hypergraph and the input feature vectors of nodes in the learned latent space under a *fully unsupervised setting*.

Figure 2 illustrates the overall HRL process in HYPERGC, consisting of three main steps: (a) hypergraph encoding, (b) hyperedge generation, and (c) hyperedge evaluation. *Hypergraph encoding* is the step of representing nodes and hyperedges of a given hypergraph into vectors in F' -dimensional latent space using contrastive learning (Figure. 2-(a)). *Hyperedge generation* creates hyperedges based on the node vectors obtained from the hypergraph encoding step by using the progressive-selection approach (Figure. 2-(b)). *Hyperedge evaluation* then assesses the quality of the generated hyperedges by using the contrastive learning-based evaluation (Figure. 2-(c)). HYPERGC performs these three steps to refine the node vectors progressively in the iterative way. We describe each step in detail in the following subsections.

4.2 Hypergraph Encoding

In the hypergraph encoding step, HYPERGC leverages contrastive learning [25, 39] to map the nodes and hyperedges of a given hypergraph into meaningful vectors. Following the encoding strategy of [25], HYPERGC first generates two distinct augmented views, $\mathcal{H}_1 = (\mathbf{X}_1, \mathbf{H}_1)$ and $\mathcal{H}_2 = (\mathbf{X}_2, \mathbf{H}_2)$, from the original hypergraph $\mathcal{H} = (\mathbf{X}, \mathbf{H})$ by applying two stochastic augmentation functions respectively. Specifically, HYPERGC augments the hypergraph using a combination of random node feature masking and membership masking [25, 43]. Subsequently, the hypergraph encoder $f_\theta(\cdot)$ produces node and hyperedge embeddings through a two-stage aggregation strategy [4, 8]. Formally, the node and hyperedge vectors at the k -th layer, $\mathbf{P}^{(k)}$ and $\mathbf{Q}^{(k)}$, are defined as:

$$\mathbf{Q}^{(k)} = \sigma \left(\mathbf{D}_E^{-1} \mathbf{H}^\top \mathbf{P}^{(k-1)} \boldsymbol{\Theta}_E^{(k)} + \mathbf{b}_E^{(k)} \right), \quad (1)$$

$$\mathbf{P}^{(k)} = \sigma \left(\mathbf{D}_V^{-1} \mathbf{H} \mathbf{W} \mathbf{Q}^{(k)} \boldsymbol{\Theta}_V^{(k)} + \mathbf{b}_V^{(k)} \right), \quad (2)$$

where $\mathbf{P}^{(0)} = \mathbf{X}$ and $\mathbf{W}$ is the identity matrix. $\boldsymbol{\Theta}_E^{(k)}$ and $\boldsymbol{\Theta}_V^{(k)}$ denote trainable weight matrices; $\mathbf{b}_E^{(k)}$ and $\mathbf{b}_V^{(k)}$ are the corresponding trainable bias terms, and σ represents the non-linear activation function, PReLU [14].

4.3 Hyperedge Generation

In the hyperedge generation step, HYPERGC leverages a progressive selection strategy to reconstruct a set of target hyperedges $\tilde{\mathbf{E}} = \{\tilde{\mathbf{e}}_1, \dots, \tilde{\mathbf{e}}_n\}$, producing $\hat{\mathbf{E}} = \{\hat{\mathbf{e}}_1, \dots, \hat{\mathbf{e}}_n\}$ based on the node vectors $\mathbf{P}$ obtained during the encoding step. For each view, the hyperedge generator $h_\psi(\cdot)$ begins by randomly selecting a single seed node from a target hyperedge $\tilde{\mathbf{e}}_j$ to initialize the reconstruction of $\hat{\mathbf{e}}_j$. Starting from this seed node, HyperGC progressively adds to the hyperedge $\hat{\mathbf{e}}_j$ the most relevant node at each iteration, chosen from the candidates within a predefined hop range (min–max), until the size of hyperedge $\hat{\mathbf{e}}_j$ reaches that of $\tilde{\mathbf{e}}_j$.

To identify the most relevant node, the hyperedge generator $h_\psi(\cdot)$ employs a *recurrent model* combined with an attention mechanism, which is to capture contextual dependencies among a set of previously selected nodes. In our case, we adopt an Long Short-Term Memory (LSTM) as the recurrent model, as it achieved the best performance among the considered alternatives i.e., RNN and GRU (see the results in Section 5.2-EQ3 and Table 11 of Appendix A.4). Specifically, the embeddings of selected nodes are sequentially fed into the LSTM to produce a context vector $\mathbf{h}_t$, which is then used as a query in an attention operation over a subset of candidate node embeddings $\mathbf{P}_n$ (serving as keys) within a predefined min–max hop range from the seed node. The node with the highest attention score is selected as the most relevant one in the current context and appended to $\hat{\mathbf{e}}_j$. This process is repeated until the size of $\hat{\mathbf{e}}_j$ matches that of the target hyperedge $\tilde{\mathbf{e}}_j$. The same reconstruction procedure is applied to all hyperedges in the view. Accordingly, the number of LSTM cells is set to the maximum hyperedge cardinality, ensuring sufficient capacity to generate all hyperedges. In Section 5.2, we further analyze the effects of four key factors in hyperedge generation: (1) the recurrent model, (2) the seed node selection strategy, (3) the candidate node range, and (4) the number of generated hyperedges.

4.4 Hyperedge Evaluation

In the hyperedge evaluation step, HYPERGC assesses the quality of the generated hyperedges $\hat{\mathbf{e}}_j$ using a contrastive learning-based evaluation strategy. To this end, we design a loss function that jointly considers two aspects: (1) the *correctness of node selection* in the hyperedge generation, and (2) the *structural consistency* of the generated hyperedges with respect to the target hyperedge. This involves not only verifying whether each selected node belongs to the ground-truth set of nodes in the target hyperedge but also ensuring that the overall configuration of selected nodes retains the relational semantics and higher-order connectivity embedded in the structure of the original hypergraph.

Regarding the first aspect, i.e., correctness of node selection, it is important to note that node selection for hyperedge reconstruction does not rely on any predefined order—what matters is that all nodes in the target hyperedge are eventually selected, regardless of the selection order. At each iteration, if the selected node belongs to the target hyperedge, it is regarded as a correctly reconstructed node and removed from the set of remaining nodes in the target hyperedge that need to be reconstructed in the following iterations. Otherwise, the current nodes belonging to the target hyperedge remain unchanged. As a result, the supervision signal (i.e., a set of current nodes in a target hyperedge) evolves dynamically. This set is continuously updated and gradually reduced over time as correct selections are made. Consequently, HYPERGC should adapt its selection strategy in response to this evolving signal, rather than relying on a fixed signal. This dynamic nature introduces additional challenges, requiring our HYPERGC to remain both flexible and context-aware throughout the progressive selection process.

To this end, we propose a dynamic target cross-entropy loss (DTCE). We first represent the target hyperedge $\tilde{\mathbf{e}}_j$ as a ground-truth multi-hot vector $\mathcal{Y}^{(0)}$, where nodes belonging to $\tilde{\mathbf{e}}_j$ are assigned a value of 1 and all others are set to 0. At each iteration t , the target vector $\mathcal{Y}^{(t)}$ is updated by removing the selected node—i.e., changing its entry from 1 to 0—if the node belongs to the target hyperedge, as defined below:

$$\mathcal{Y}_i^{(t)} = \begin{cases} 0, & \text{if } i = i^* \text{ and } \mathcal{Y}_i^{(t-1)} = 1, \\ \mathcal{Y}_i^{(t-1)}, & \text{otherwise,} \end{cases} \quad (3)$$

where i indexes the target vector $\mathcal{Y}$, and $i^* = \arg \max_k \hat{p}_k^{(t-1)}$ denotes the index of the selected node, with $\hat{p}_k^{(t-1)}$ being the predicted probability of node v_k belonging to the target hyperedge. Intuitively, DTCE removes correctly selected nodes from the remaining target set at each iteration.

We formulate the DTCE loss $\mathcal{L}_{ce}$ as the cross-entropy between the predicted probability distribution over candidate nodes and the dynamically updated target at iteration t , which is optimized by HYPERGC at each iteration to progressively reconstruct the target hyperedge, given by:

$$\mathcal{L}_{ce}(\hat{\mathbf{E}}) = - \sum_{\hat{\mathbf{e}}_j \in \hat{\mathbf{E}}} \sum_{t=1}^T \sum_{i=1}^N \left[\mathcal{Y}_i^{(t)} \cdot \log \left(\hat{p}_i^{(t)} + \epsilon \right) + \left(1 - \mathcal{Y}_i^{(t)} \right) \cdot \log \left(1 - \hat{p}_i^{(t)} + \epsilon \right) \right], \quad (4)$$

where T and N denote the number of LSTM steps (i.e., the target hyperedge size to reconstruct) and the total number of nodes in the hypergraph, respectively.

To evaluate the second aspect, i.e., structural consistency, HYPERGC leverages contrastive learning to align hyperedge vectors generated from two different views. Specifically, for a hyperedge $\hat{\mathbf{e}}_j$ generated in one view, if there exists a corresponding hyperedge $\tilde{\mathbf{e}}_j$ in the other view that contains the identical set of nodes, the pair is treated as a positive pair; if two hyperedges in the two views contain different sets of nodes, they are regarded as a negative pair. To comprehensively capture structural semantics, HYPERGC performs contrastive learning at both the *node-level*—aligning individual node vectors across views—and the *hyperedge-level*—aligning vectors of node sets forming hyperedges.

More specifically, for a node-level contrast, HYPERGC uses the vector of node v_i from the first view, $\mathbf{z}_{1,i}$, as the anchor; its counterpart from the second view, $\mathbf{z}_{2,i}$, as the positive sample, and *all* other node vectors $\mathbf{z}_{2,k}$, where $k \neq i$, as negative samples. The loss for each node v_i is defined as:

$$\ell_n(\mathbf{z}_{1,i}, \mathbf{z}_{2,i}) = -\log \frac{\exp(s(\mathbf{z}_{1,i}, \mathbf{z}_{2,i})/\tau_n)}{\sum_{k=1}^{|V|} \exp(s(\mathbf{z}_{1,i}, \mathbf{z}_{2,k})/\tau_n)}, \quad (5)$$

where τ_n is a temperature parameter [25], and $s(\cdot, \cdot)$ is a scoring function (*i.e.*, cosine similarity) that assigns a high score to positive pairs and a low score to negative pairs. Finally, the average loss over all nodes across the two (potentially asymmetric) views is computed as the overall node-level contrastive loss as:

$$\mathcal{L}_n = \frac{1}{2|V|} \sum_{i=1}^{|V|} \{\ell_n(\mathbf{z}_{1,i}, \mathbf{z}_{2,i}) + \ell_n(\mathbf{z}_{2,i}, \mathbf{z}_{1,i})\}. \quad (6)$$

For a hyperedge-level contrast, HYPERGC treats $\mathbf{y}_{1,j}$ of hyperedge e_j as the anchor, its counterpart $\mathbf{y}_{2,j}$ as the positive sample, and all other vectors $\mathbf{y}_{2,k}$ ($k \neq j$) as negatives. A positive pair is formed by two views of the same hyperedge: although membership masking yields divergent node compositions, both $\mathbf{y}_{1,j}$ and $\mathbf{y}_{2,j}$ inherit the original index j from the incidence matrix. The contrastive loss for e_j is defined as:

$$\ell_h(\mathbf{y}_{1,j}, \mathbf{y}_{2,j}) = -\log \frac{\exp(s(\mathbf{y}_{1,j}, \mathbf{y}_{2,j})/\tau_h)}{\sum_{k=1}^{|E|} \exp(s(\mathbf{y}_{1,j}, \mathbf{y}_{2,k})/\tau_h)}, \quad (7)$$

where τ_h is a temperature parameter [25]. The average loss over all hyperedges across the two (potentially asymmetric) views is computed as the overall hyperedge-level contrastive loss as:

$$\mathcal{L}_h = \frac{1}{2|E|} \sum_{j=1}^{|E|} \{\ell_h(\mathbf{y}_{1,j}, \mathbf{y}_{2,j}) + \ell_h(\mathbf{y}_{2,j}, \mathbf{y}_{1,j})\}. \quad (8)$$

By integrating Eq. 6 and Eq. 8, our contrastive loss is formulated as:

$$\mathcal{L}_{cl} = \omega_n \mathcal{L}_n + \omega_h \mathcal{L}_h. \quad (9)$$

Finally, the overall loss of HYPERGC, which jointly evaluates the two aspects (*i.e.*, correctness of node selection and structural consistency), is defined as follows:

$$\mathcal{L} = \omega_{ce} \mathcal{L}_{ce} + \omega_{cl} \mathcal{L}_{cl}. \quad (10)$$

Through this joint evaluation strategy, HYPERGC is guided to effectively capture structural semantics by aligning node and hyperedge vectors across views within a coherent vector space. In this space, node vectors belonging to the same hyperedge are getting close, while those from different hyperedges are getting distant, thereby promoting clear separation of higher-order relationships.

4.5 Discussions

Hyperedges represent groupwise relationships among multiple nodes and are therefore naturally interpreted as *unordered sets*, where the semantic meaning is invariant to node ordering. Meanwhile, HYPERGC generates hyperedges progressively using a recurrent model, in which nodes are selected *sequentially*, potentially introducing an implicit ordering during the generation. This ordering is *not intended to alter the semantics of hyperedges*, but rather serves as a *modeling mechanism* to capture contextual information from previously selected nodes. Therefore, the final generated

hyperedge remains a *permutation-invariant set*. We validate empirically observe that different generation orders lead to no significant differences in downstream task performance (See Appendix A.2).

Additionally, in HYPERGC, the number of LSTM cells is set to the size of the largest hyperedge, ensuring sufficient expressive capacity. However, this may cause substantial computational and memory overhead when hyperedge sizes are large or highly variable. To improve training efficiency, we instead configure the number of cells based on quantiles of the hyperedge size distribution, specifically using the third quartile (Q3, 75%) or the median (Q2, 50%). Our results show that this quantile-based strategy reduces training time by about 50% with no significant loss in downstream task performance, achieving a more favorable efficiency–performance trade-off (See Section 5.2-EQ5).

5 Evaluation

In this section, we validate the effectiveness of HYPERGC through extensive experiments. Our experiments are designed to answer the following evaluation questions (EQs):

- **EQ1:** Are the proposed strategies (S1) and (S2) effective, both individually and in combination?
- **EQ2:** Does HYPERGC outperform existing state-of-the-art HRL methods across various downstream tasks?
- **EQ3:** How do different design choices in hyperedge generation affect the performance of HYPERGC?
- **EQ4:** How sensitive is HYPERGC to key hyperparameters?
- **EQ5:** How efficient and scalable is HYPERGC with respect to computational overhead and training scalability?

5.1 Experimental Settings

Datasets. In this paper, we use 11 real-world hypergraph datasets collected from diverse domains, including co-citation (Cora [42], Citeseer [42], and PubMed [42]), co-authorship (Cora-CA [42], DBLP-P [42], DBLP-A [37], and AMiner [45]), computer graphics (MN-40 [41]), movie–actor relationships (IMDB [37]), news content (20News [21]), and political membership affiliations (House [3]).¹ We note that these datasets form the full union of those adopted in prior work [4, 17, 22, 25, 42]. Here, we summarize the basic statistics of these datasets in Table 1.

Competing methods. We compare HYPERGC with the following 19 competing methods. They include (1) 9 state-of-the-art *semi-supervised* HRL methods, *i.e.*, HGNN [10], HyperGCN [42], HNNH [8], UniGNN variants, *i.e.*, UniGCN, UniGIN, UniGCNII [17], AllSet [3], ED-HNN [36], and PhenomNN [38]; (2) 6 state-of-the-art *self-supervised* HRL methods, *i.e.*, TriCL [25], HyperGCL [39], HyperGRL [9], HypeBoy [22], Villain [26], and SE-HSSL [27]; (3) 3 state-of-the-art self-supervised representation learning methods for conventional graphs, *i.e.*, GraphMAE2 [16], MaskGAE [28], and GGD [47]; (4) 1 baseline method, *i.e.*, MLP [12]. Note that GraphMAE2, MaskGAE, GGD are originally designed for conventional graphs, rather than hypergraphs. To evaluate these models fairly, we adopt the same experimental setup as described in [22]. Specifically, for GGD, we replace the original feature and topology augmentations with the transformations proposed in [22]. For GraphMAE2

¹We do not include the datasets used in Section 3 in our final evaluation, as it lacks ground-truth node labels; instead, we employ eleven labeled datasets.

Table 1: Statistics of eleven real-world hypergraphs used in our experiments

	Citeseer	Cora	DBLP-A	Cora-CA	IMDB	House	Pubmed	DBLP-P	AMiner	MN-40	20News
# of Nodes	1,458	1,434	2,591	2,388	3,939	1,290	3,840	41,302	20,201	12,311	16,242
# of Hyperedges	1,079	1,579	2,690	1,072	2,015	341	7,963	22,263	8,052	12,311	100
# of Classes	6	7	4	7	3	2	3	6	12	40	4
# of Features	3,703	1,433	334	1,433	3,066	2	500	1,425	500	100	100

and MaskGAE, we first expand the input hypergraph into a graph by using *clique expansion* [32], and then apply each model to the expanded graph. Additionally, since MLP does not utilize hypergraph topology, we directly use the node feature vectors and their corresponding labels for training, and subsequently perform downstream tasks.

Target downstream tasks and evaluation protocols. To evaluate HYPERGC across downstream tasks, we conduct node classification [22, 39], hyperedge prediction [22, 25], and community detection [25]. Due to space limitations, except for the comparison with competing methods, only node classification results are presented here, while the remaining results are reported in Appendices A.4. For node classification, following [22, 39], we randomly split nodes into training, validation, and test sets with ratios of 1%, 1%, and 98%, respectively, and report the average and standard deviation of accuracy over 20 random splits. Each self-supervised HRL method first learns node representations, followed by training an MLP classifier [22]. For hyperedge prediction, following [22, 39], we randomly split nodes into training, validation, and test sets with ratios of 60%, 20%, and 20%, respectively, and report AUROC over 20 random splits. Negative hyperedges are generated using Size-Negative-Sampling [31], and hyperedge representations are obtained via max-min pooling [22, 42]. For community detection, following [25], we apply k -means [19] to learned node representations and evaluate performance using NMI [6]. To ensure fairness, only self-supervised HRL methods are compared, and the number of clusters is set to the number of ground-truth communities.

Reproducibility. For reproducibility, we release our source code at <https://github.com/cowbean-i/HyperGC> and provide detailed experimental settings, including the hyperparameters of HYPERGC, in Appendix A.3.

5.2 Experimental Results

EQ1: Ablation study. In this experiment, we individually assess the effectiveness of the strategies proposed in HYPERGC. Specifically, we compare its five variants: **(V1)** HYPERGC without employing neither full reconstruction (FR) nor contrastive learning-based evaluation (CL); **(V2)** HYPERGC with CL only; **(V3)** HYPERGC with FR (without DTCE) only; **(V4)** HYPERGC with employing both FR (without DTCE) and CL; and **(V5)** HYPERGC with employing both FR (with DTCE) and CL (*i.e.*, our final choice for HYPERGC). Note that **(V1)** simulates an existing generative HRL method, in which partial reconstruction and discrimination-based evaluation are implemented by using our encoder.

Table 2 shows the results of our ablation study. For each dataset, the best and second-best performances are marked in **bold** and underline, respectively. OOT (out of time) denotes the results not obtained within 24 hours, and OOM (out of memory) indicates memory shortage errors on a 48GB GPU. We summarize the node classification results as follows. First, variant **(V5)** incorporating both

strategies consistently yields the highest accuracy in *all* datasets. Second, the improvement of **(V2)** over **(V1)** demonstrates the effectiveness of contrastive evaluation in addressing challenge **(C2)**. Moreover, the additional performance gains of **(V5)** over **(V2)** highlight the importance of full hyperedge reconstruction with DTCE for challenge **(C1)**. Moreover, variant **(V5)** incorporating DTCE consistently outperforms its counterpart **(V4)** without DTCE. Notably, **(V3)** often performs worse than baseline **(V1)**, which uses no strategy at all. This highlights the necessity of modeling dynamic supervision signals, especially for progressive full hyperedge reconstruction.

EQ2: Accuracy comparison. Next, we evaluate the performance of HYPERGC against 19 competing methods introduced in Section 5.1. Table 3 shows the results for node classification. For each dataset, the best performance among *all methods* (including HYPERGC) and the best performance among *existing competing methods* (which typically corresponds to the second-best overall) are highlighted in **bold** and underline, respectively. *AR* denotes the *average rank* across all datasets, where a lower value indicates better overall performance. HYPERGC₅₀, HYPERGC₇₅, and HYPERGC₁₀₀ denote the variants of HYPERGC in which the number of LSTM cells is set to the median, the third quartile (Q3), and the maximum hyperedge size, respectively. Results of the HYPERGC variants are reported only for EQ2 and EQ5; in all other experiments, we report only HYPERGC₁₀₀, referred to as HYPERGC, for brevity.

We summarize the results in Table 3 as follows. First, HYPERGC outperforms all competing methods, achieving the highest accuracy on 10 out of 11 datasets and the lowest average rank (1.2). In particular, it achieves substantial improvements over leading self-supervised methods such as HypeBoy (6.1), TriCL (6.9), SE-HSSL (9.4), and ViLain (17.4), with notable gains on IMDB (+12.27%), Cora (+7.84%), and Citeseer (+5.89%), while the overall performance trends among competing methods remain similar to those reported in prior studies [29, 30]. Second, although the two efficient variants (*i.e.*, HYPERGC₅₀ and HYPERGC₇₅) of HYPERGC achieve slightly lower accuracy than the full setting (HYPERGC₁₀₀), the performance gap remains small. Notably, these variants still outperform most competing methods across most datasets. Their computational overhead is further discussed in EQ5. Third, our HYPERGC outperforms other generative HRL methods (*i.e.*, HypeBoy and HyperGRL), likely because it explicitly models the real-world hyperedge formation process via full reconstruction. Lastly, self-supervised HRL methods generally outperform semi-supervised ones across diverse datasets, demonstrating their effectiveness in capturing complex groupwise relationships without relying on labeled data. Similar trends are also observed in hyperedge prediction and community detection, as shown in Tables 4 and 5, respectively.

Table 2: Ablation study: Accuracy comparison of 4 variants of HYPERGC with node classification on 11 real-world hypergraphs

	FR	CL	Citeseer	Cora	DBLP-A	Cora-CA	IMDB	House	Pubmed	DBLP-P	AMiner	MN-40	20News	AR
V1	✗	✗	33.2 ± 9.1	40.6 ± 6.1	56.3 ± 5.5	44.5 ± 9.9	35.4 ± 2.8	67.9 ± 4.5	68.7 ± 4.3	89.3 ± 0.4	30.8 ± 1.7	<u>95.6 ± 0.7</u>	72.7 ± 1.9	3.7
V2	✗	✓	<u>51.7 ± 8.2</u>	<u>57.9 ± 10.2</u>	<u>81.3 ± 2.4</u>	<u>65.9 ± 4.5</u>	51.4 ± 4.5	67.3 ± 4.4	75.2 ± 4.3	89.3 ± 0.4	<u>34.3 ± 2.8</u>	95.5 ± 1.3	<u>77.6 ± 0.7</u>	2.7
V3 (w/o DTCE)	✓	✗	49.3 ± 9.8	52.8 ± 7.2	73.4 ± 3.8	41.8 ± 8.2	<u>51.7 ± 4.0</u>	<u>68.9 ± 4.7</u>	75.5 ± 2.8	<u>86.7 ± 0.5</u>	30.0 ± 1.9	95.4 ± 1.0	76.5 ± 0.7	3.5
V4 (w/o DTCE)	✓	✓	46.6 ± 11.7	51.0 ± 10.4	65.3 ± 4.3	43.2 ± 9.6	51.7 ± 4.3	67.9 ± 4.6	<u>76.8 ± 3.8</u>	85.5 ± 0.7	29.7 ± 1.6	95.6 ± 0.9	76.5 ± 0.8	3.6
V5 (w DTCE)	✓	✓	61.1 ± 13.0	67.4 ± 7.4	82.5 ± 3.4	69.3 ± 2.7	54.6 ± 2.9	69.8 ± 4.0	78.5 ± 2.5	89.3 ± 0.4	35.1 ± 2.1	95.6 ± 0.8	77.6 ± 0.4	1.1

Table 3: Accuracy comparison of HYPERGC and 19 competing methods with node classification on 11 real-world hypergraphs

		Citeseer	Cora	DBLP-A	Cora-CA	IMDB	House	Pubmed	DBLP-P	AMiner	MN-40	20News	AR
Semi-supervised	MLP	32.4 ± 8.1	29.4 ± 5.3	56.6 ± 5.4	36.0 ± 4.7	37.6 ± 3.3	73.1 ± 3.2	62.8 ± 3.0	74.6 ± 0.8	22.7 ± 1.8	88.5 ± 1.5	73.3 ± 1.3	17.0
	HGNN	38.1 ± 10.7	45.1 ± 9.1	65.0 ± 5.7	44.3 ± 8.0	41.5 ± 2.8	51.9 ± 4.7	70.8 ± 3.8	84.0 ± 0.9	29.7 ± 2.0	89.5 ± 1.3	67.1 ± 2.0	14.3
	HyperGCN	33.8 ± 7.3	39.1 ± 8.2	64.3 ± 6.3	45.0 ± 9.5	40.6 ± 3.1	48.2 ± 0.9	59.3 ± 15.2	83.6 ± 1.6	26.8 ± 2.2	57.8 ± 11.6	71.7 ± 2.1	17.7
	HNHN	44.2 ± 7.9	52.5 ± 7.3	69.0 ± 4.5	53.1 ± 7.4	42.5 ± 3.6	56.7 ± 4.0	69.1 ± 3.4	86.4 ± 0.8	32.1 ± 2.7	89.2 ± 1.4	74.4 ± 1.1	10.9
	UniGCN	39.8 ± 9.2	43.1 ± 6.6	59.4 ± 5.1	46.3 ± 6.6	41.0 ± 3.0	51.7 ± 2.7	67.6 ± 5.6	82.2 ± 0.9	29.8 ± 1.8	75.7 ± 11.4	67.4 ± 2.5	16.0
	UniGIN	41.4 ± 10.3	45.5 ± 7.9	63.1 ± 5.5	49.2 ± 6.9	41.7 ± 2.7	50.8 ± 2.7	70.4 ± 4.9	83.5 ± 0.8	30.8 ± 1.2	87.1 ± 2.3	68.1 ± 2.4	14.1
	UniGCN2	39.6 ± 10.7	43.9 ± 9.5	58.7 ± 7.9	55.3 ± 5.3	41.6 ± 3.9	58.8 ± 10.1	72.6 ± 3.8	86.1 ± 0.6	32.3 ± 1.5	79.7 ± 6.0	76.8 ± 1.4	12.2
	AllSet	41.1 ± 9.8	49.0 ± 8.1	64.9 ± 4.6	53.6 ± 8.2	41.7 ± 2.9	50.3 ± 3.1	74.1 ± 5.0	85.6 ± 0.7	29.8 ± 3.2	89.4 ± 1.3	<u>77.2 ± 0.9</u>	11.5
	ED-HNN	32.9 ± 11.6	29.3 ± 3.9	58.3 ± 5.2	36.3 ± 8.7	37.2 ± 2.6	<u>71.0 ± 5.4</u>	61.9 ± 3.3	85.1 ± 1.1	27.1 ± 2.8	73.0 ± 3.8	75.7 ± 1.2	16.4
	PhenomNN	42.2 ± 9.0	55.8 ± 8.0	70.3 ± 5.3	56.2 ± 6.6	42.1 ± 2.6	69.4 ± 7.0	<u>76.8 ± 3.4</u>	O.O.M	O.O.M	94.0 ± 0.5	O.O.T	7.8
Self-supervised	GraphMAE2	51.7 ± 12.2	61.0 ± 9.1	77.2 ± 2.4	64.3 ± 6.3	45.6 ± 3.1	52.4 ± 3.0	72.6 ± 4.4	87.4 ± 0.4	34.7 ± 2.3	90.6 ± 1.2	71.8 ± 0.4	8.3
	MaskGAE	53.2 ± 10.8	55.8 ± 9.0	78.2 ± 2.7	59.8 ± 4.1	45.0 ± 3.0	53.0 ± 5.1	75.4 ± 3.2	86.9 ± 0.5	33.8 ± 1.7	91.0 ± 0.8	O.O.T	7.7
	GGD	34.0 ± 5.8	37.2 ± 6.7	58.8 ± 6.5	32.2 ± 6.3	37.6 ± 3.2	50.6 ± 1.8	64.9 ± 4.4	50.5 ± 6.4	31.5 ± 1.7	76.8 ± 2.0	47.5 ± 7.9	17.9
	TriCL	53.0 ± 9.9	61.5 ± 5.3	80.3 ± 3.5	63.4 ± 3.9	47.5 ± 4.6	65.2 ± 5.3	74.0 ± 5.2	87.6 ± 0.5	34.6 ± 2.7	93.0 ± 0.8	73.8 ± 1.8	6.9
	HyperGCL	43.4 ± 8.4	54.8 ± 5.6	69.5 ± 3.7	61.8 ± 3.0	48.4 ± 2.5	63.7 ± 5.0	71.0 ± 2.2	O.O.M	30.4 ± 1.8	<u>94.6 ± 1.2</u>	74.0 ± 1.9	9.2
	HyperGRL	35.1 ± 7.6	37.0 ± 5.3	41.1 ± 5.1	41.8 ± 5.8	35.7 ± 4.2	50.4 ± 2.2	50.2 ± 8.7	78.7 ± 1.2	28.0 ± 2.8	89.4 ± 1.5	O.O.T.	18.3
	HypeBoy	<u>57.7 ± 10.8</u>	61.9 ± 8.5	<u>81.2 ± 2.7</u>	<u>67.0 ± 3.7</u>	48.3 ± 5.0	67.7 ± 6.5	73.7 ± 4.1	<u>87.9 ± 0.5</u>	34.6 ± 2.8	89.2 ± 2.2	75.7 ± 2.1	6.1
	Villain	33.6 ± 9.4	47.1 ± 8.4	40.4 ± 3.9	31.4 ± 4.1	39.7 ± 4.4	50.6 ± 1.8	73.7 ± 3.3	69.0 ± 0.9	19.9 ± 1.2	69.7 ± 2.3	74.3 ± 3.0	17.4
	SE-HSSL	57.0 ± 8.2	<u>62.5 ± 6.2</u>	77.1 ± 2.7	63.9 ± 5.0	<u>48.9 ± 4.1</u>	48.0 ± 2.0	69.8 ± 2.8	83.6 ± 0.9	<u>34.8 ± 4.2</u>	85.8 ± 5.2	75.8 ± 1.0	9.4
	HYPERGC₅₀	61.0 ± 13.0	66.5 ± 7.5	81.8 ± 3.2	67.5 ± 5.8	54.3 ± 2.5	69.2 ± 3.8	77.5 ± 2.8	89.0 ± 0.5	35.1 ± 2.5	95.6 ± 0.8	77.4 ± 0.7	2.7
	HYPERGC₇₅	58.9 ± 13.1	64.9 ± 9.5	82.4 ± 2.5	68.2 ± 5.8	54.4 ± 2.4	68.6 ± 4.5	78.2 ± 2.5	89.2 ± 0.4	34.9 ± 2.5	95.6 ± 0.8	77.6 ± 0.4	2.5
	HYPERGC₁₀₀	61.1 ± 13.0	67.4 ± 7.4	82.5 ± 3.4	69.3 ± 2.7	54.6 ± 2.9	69.8 ± 4.0	78.5 ± 2.5	89.3 ± 0.4	35.1 ± 2.1	95.6 ± 0.8	77.6 ± 0.4	1.2
	Improvement	5.89%	7.84%	1.60%	3.28%	12.27%	-4.51%	2.21%	1.59%	1.15%	1.06%	0.52%	

Table 4: Accuracy comparison of HyperGC and 19 competing methods with hyperedge prediction on 11 real-world hypergraphs

		Citeseer	Cora	DBLP-A	Cora-CA	IMDB	House	Pubmed	DBLP-P	AMiner	MN-40	20News	AR
Semi-supervised	MLP	65.6 ± 2.0	77.5 ± 1.5	74.2 ± 1.5	58.6 ± 2.9	42.0 ± 2.2	54.8 ± 5.3	88.3 ± 0.6	90.2 ± 0.5	82.3 ± 1.0	94.9 ± 8.4	95.1 ± 3.9	12.5
	HGNN	54.8 ± 4.1	57.6 ± 6.0	64.7 ± 1.3	65.0 ± 4.6	<u>60.5 ± 1.9</u>	50.4 ± 1.8	65.9 ± 2.1	61.0 ± 8.4	69.6 ± 1.2	50.2 ± 0.6	49.4 ± 8.2	15.5
	HyperGCN	54.9 ± 2.1	64.9 ± 2.0	66.5 ± 1.2	49.3 ± 1.5	48.4 ± 1.9	87.7 ± 2.5	74.1 ± 0.6	53.9 ± 2.1	50.8 ± 2.1	60.9 ± 3.0	55.6 ± 15.6	15.6
	HNHN	53.7 ± 2.5	59.0 ± 1.8	62.4 ± 1.9	51.6 ± 1.4	48.2 ± 0.7	69.7 ± 5.4	65.7 ± 2.4	51.2 ± 1.6	48.1 ± 1.3	61.0 ± 4.3	56.0 ± 9.7	17.3
	UniGCN	54.7 ± 3.2	51.6 ± 2.0	51.8 ± 2.1	60.0 ± 5.9	53.7 ± 4.1	50.0 ± 0.0	52.1 ± 3.0	67.0 ± 1.5	69.0 ± 1.3	51.2 ± 2.4	50.0 ± 0.0	17.7
	UniGIN	52.9 ± 3.0	63.1 ± 3.7	54.0 ± 3.0	58.7 ± 8.2	54.6 ± 4.2	50.0 ± 0.0	51.5 ± 3.6	69.2 ± 0.6	69.1 ± 1.4	50.4 ± 1.7	50.0 ± 0.0	17.4
	UniGCN2	60.2 ± 2.1	63.5 ± 3.5	61.6 ± 1.7	51.9 ± 4.3	57.5 ± 7.0	50.0 ± 0.0	69.1 ± 1.2	60.2 ± 2.5	45.9 ± 4.9	51.7 ± 1.9	50.0 ± 0.0	16.3
	AllSet	51.8 ± 3.2	54.5 ± 3.4	60.5 ± 3.0	51.2 ± 3.2	50.3 ± 1.0	53.5 ± 5.1	52.3 ± 1.6	53.2 ± 1.5	51.0 ± 1.8	56.4 ± 1.6	50.1 ± 0.4	18.4
	ED-HNN	58.0 ± 3.8	72.2 ± 2.1	61.9 ± 6.4	58.9 ± 2.1	49.4 ± 1.2	50.6 ± 1.8	53.2 ± 2.7	51.2 ± 1.6	56.0 ± 3.7	52.7 ± 1.6	51.4 ± 3.5	16.5
	PhenomNN	54.5 ± 2.7	56.3 ± 1.4	64.6 ± 2.0	50.3 ± 1.9	49.7 ± 0.9	50.8 ± 2.0	64.0 ± 4.6	O.O.M	O.O.M	63.5 ± 8.6	O.O.T	17.5
Self-supervised	GraphMAE2	78.2 ± 2.3	82.0 ± 1.4	83.3 ± 1.4	74.7 ± 2.7	47.2 ± 1.0	71.6 ± 5.5	93.6 ± 2.6	92.7 ± 0.3	78.1 ± 0.8	<u>93.8 ± 0.0</u>	87.0 ± 3.4	10.1
	MaskGAE	86.7 ± 1.8	86.7 ± 1.6	87.3 ± 1.3	76.5 ± 2.1	54.4 ± 1.7	88.0 ± 3.7	95.5 ± 0.4	94.8 ± 0.4	87.6 ± 0.9	99.9 ± 0.0	O.O.T	5.8
	GGD	72.2 ± 5.0	71.9 ± 3.1	81.4 ± 1.9	73.2 ± 4.0	53.1 ± 1.8	87.9 ± 3.1	87.2 ± 0.7	91.6 ± 1.0	84.9 ± 2.1	99.5 ± 0.1	87.9 ± 3.1	10.0
	TriCL	90.5 ± 1.2	90.7 ± 1.3	<u>87.9 ± 1.4</u>	<u>87.8 ± 1.5</u>	58.9 ± 2.1	90.0 ± 2.6	91.9 ± 0.5	94.8 ± 0.2	90.4 ± 0.6	99.6 ± 0.1	98.2 ± 3.0	4.3
	HyperGCL	73.9 ± 2.6	85.4 ± 1.5	83.5 ± 1.0	81.1 ± 1.9	53.8 ± 2.4	76.3 ± 6.3	89.6 ± 0.5	83.6 ± 0.6	82.1 ± 7.6	99.4 ± 0.1	76.3 ± 6.3	9.8
	HyperGRL	83.2 ± 8.0	84.0 ± 1.7	80.8 ± 1.4	80.0 ± 3.3	54.7 ± 2.8	<u>88.2 ± 3.3</u>	78.3 ± 1.8	88.2 ± 0.3	81.5 ± 0.7	98.8 ± 0.4	O.O.T	9.3
	HypeBoy	86.4 ± 2.1	91.1 ± 1.2	87.0 ± 1.2	87.3 ± 1.6	59.4 ± 1.9	87.2 ± 3.0	92.1 ± 0.5	<u>95.2 ± 0.2</u>	<u>89.7 ± 0.6</u>	99.7 ± 0.0	97.2 ± 3.2	5.7
	Villain	58.3 ± 2.7	68.5 ± 1.9	57.3 ± 2.0	64.4 ± 3.2	43.9 ± 2.4	77.2 ± 5.0	83.7 ± 0.7	56.3 ± 0.7	62.7 ± 1.3	79.8 ± 0.7	59.2 ± 7.8	14.7
	SE-HSSL	<u>90.5 ± 1.1</u>	<u>91.9 ± 1.3</u>	87.4 ± 1.2	85.6 ± 2.1	55.6 ± 2.0	80.4 ± 3.0	94.5 ± 0.7	94.0 ± 0.4	87.5 ± 0.6	99.8 ± 0.1	95.2 ± 5.8	6.1
	HYPERGC₅₀	90.8 ± 1.3	92.7 ± 1.2	90.9 ± 1.1	88.4 ± 1.4	63.5 ± 2.6	87.9 ± 2.7	95.8 ± 0.3	96.2 ± 0.1	89.2 ± 1.8	99.9 ± 0.0	97.4 ± 2.1	2.7
	HYPERGC₇₅	90.3 ± 1.2	92.8 ± 1.3	91.0 ± 1.0	88.1 ± 1.4	63.5 ± 2.1	88.0 ± 3.1	95.8 ± 0.3	96.2 ± 0.1	89.1 ± 1.8	99.9 ± 0.0	97.4 ± 2.6	2.7
	HYPERGC₁₀₀	90.8 ± 1.2	92.8 ± 1.2	91.0 ± 1.0	89.3 ± 1.1	63.6 ± 2.3	88.0 ± 3.0	95.8 ± 0.4	96.3 ± 0.1	89.3 ± 1.4	99.9 ± 0.0	<u>97.5 ± 2.2</u>	1.6
	Improvement	+0.33%	+0.98%	+3.53%	+5.12%	+1.76%	-2.22%	+0.31%	+1.16%	-1.22%	0.00%	-0.41%	

Table 5: Accuracy comparison of HyperGC and 9 competing methods with community detection on 11 real-world hypergraphs

		Citeseer	Cora	DBLP-A	Cora-CA	IMDB	House	Pubmed	DBLP-P	AMiner	MN-40	20News	AR
Self-supervised	GraphMAE2	<u>42.1</u>	48.03	44.82	42.12	6.03	0.15	19.99	58.44	40.12	88.59	29.96	7.1
	MaskGAE	34.81	49.17	54.21	37.98	2.91	1.45	28.91	61.64	36.33	88.33	O.O.T	6.4
	GGD	6.32	11.26	0.62	6.00	0.28	0.05	6.68	2.61	4.41	13.93	0.00	11.3
	TriCL	38.48	<u>53.58</u>	48.70	40.86	5.51	2.86	32.71	<u>61.74</u>	40.93	92.06	35.94	5.7
	HyperGCL	22.21	39.74	51.69	30.55	1.39	<u>11.48</u>	24.21	O.O.M	31.90	94.04	38.19	7.1
	HyperGRL	26.01	36.38	23.60	34.50	1.75	0.00	27.82	57.43	0.23	51.05	O.O.T	9.7
	HypeBoy	40.90	41.55	44.88	41.91	<u>8.56</u>	0.00	30.43	60.82	42.21	81.94	37.97	6.5
	VilLain	11.3	37.72	13.56	9.73	0.07	0.09	<u>32.88</u>	40.14	16.24	74.8	<u>38.75</u>	9.3
	SE-HSSL	38.52	52.51	47.87	44.38	0.28	0.7	14.98	55.23	<u>41.83</u>	<u>94.8</u>	38.17	6.0
	HYPERGC₅₀	44.66	54.07	49.32	42.27	11.3	11.46	34.75	63.85	41.02	94.38	39.7	3.3
	HYPERGC₇₅	44.62	53.24	51.26	42.32	11.09	11.48	34.99	63.9	41.31	94.48	39.65	3.0
	HYPERGC₁₀₀	44.93	55.8	<u>52.35</u>	<u>42.83</u>	11.5	11.57	35.33	66.25	41.32	94.82	39.8	1.4
Improvement		+6.72%	+4.14%	-3.43%	-3.49%	+34.35%	0.78%	+7.45%	+7.30%	-2.11%	+0.02%	+2.71%	

Table 6: Node classification accuracy of HYPERGC according to seed node selection

	Citeseer	Cora	DBLP-A	Cora-CA	IMDB	House	Pubmed	DBLP-P	AMiner	MN-40	20News	AR
High degree	61.5 ± 11.9	66.8 ± 8.2	81.3 ± 3.6	68.4 ± 4.4	54.0 ± 3.2	68.8 ± 4.1	76.2 ± 5.4	89.3 ± 0.4	35.1 ± 2.6	95.7 ± 0.7	77.6 ± 0.5	2.1
Low degree	60.9 ± 12.4	67.0 ± 8.2	82.0 ± 2.9	69.1 ± 3.6	54.0 ± 3.4	68.0 ± 5.5	75.6 ± 6.4	89.3 ± 0.5	35.0 ± 2.5	95.8 ± 0.7	77.6 ± 0.5	2.5
Random	61.1 ± 13.0	67.4 ± 7.4	82.5 ± 3.4	69.3 ± 2.7	54.6 ± 2.9	69.8 ± 4.0	78.5 ± 2.5	89.3 ± 0.4	35.1 ± 2.1	95.6 ± 0.8	77.6 ± 0.4	1.3

Table 7: Node classification accuracy of HYPERGC according to recurrent model

	Citeseer	Cora	DBLP-A	Cora-CA	IMDB	House	Pubmed	DBLP-P	AMiner	MN-40	20News	AR
RNN	<u>60.5 ± 11.6</u>	63.7 ± 10.6	82.0 ± 4.2	69.3 ± 2.5	<u>53.6 ± 3.1</u>	<u>68.7 ± 4.7</u>	<u>78.2 ± 3.0</u>	89.1 ± 0.5	<u>35.1 ± 2.8</u>	95.7 ± 0.8	77.8 ± 0.6	2.0
GRU	58.3 ± 12.8	<u>63.8 ± 9.0</u>	82.6 ± 1.9	67.2 ± 8.1	52.9 ± 3.2	66.7 ± 5.7	76.0 ± 4.8	89.4 ± 0.3	34.3 ± 2.9	95.6 ± 0.9	77.3 ± 0.5	2.5
LSTM	61.1 ± 13.0	67.4 ± 7.4	<u>82.5 ± 3.4</u>	<u>69.3 ± 2.7</u>	54.6 ± 2.9	69.8 ± 4.0	78.5 ± 2.5	<u>89.3 ± 0.4</u>	35.1 ± 2.1	<u>95.6 ± 0.8</u>	<u>77.6 ± 0.4</u>	1.5

EQ3: Design choice validation. We investigate the impact of four key factors in hyperedge generation: (1) the type of a recurrent model, (2) the selection strategy for seed nodes, (3) the range of candidate nodes considered, and (4) the number of generated hyperedges. We begin by evaluating how the performance of HYPERGC varies under different recurrent models. To this end, we first generate hyperedges through three recurrent models: (1) recurrent neural network (RNN), (2) gated recurrent unit (GRU), and (3) long short term memory (LSTM). The remaining steps then follow the standard training pipeline of HYPERGC. We finally perform downstream tasks by using the node vectors learned by HYPERGC.

Table 7 shows the results for node classification. Due to space constraints, we present only node classification task results, while the hyperedge prediction and community detection results are reported in Table 11 of Appendix A.4. We summarize the results as follows. First, LSTM (*i.e.*, our final choice) achieves the highest accuracy on most datasets, with an AR of 1.5. Second, except for a few recurrent model-dataset pairs — 5 out of 33 (*e.g.*, RNN-DBLP, RNN/GRU/LSTM-House, and GRU-AMiner) — the three recurrent models generally outperform the best competing method (underlined in Table 3). These results once again indicate that modeling the real-world hyperedge formation process substantially improves the effectiveness of hypergraph learning.

Then, we examine the performance of HYPERGC varies under different seed node selection strategies. To this end, we generate hyperedges by using three types of seed nodes for each target hyperedge: (1) the node with the highest degree, (2) the node with the lowest degree, and (3) the node randomly selected. We then

perform downstream tasks by using the node vectors learned by HYPERGC. Table 6 shows the results for node classification. Although the differences in accuracy are relatively small, random seed selection consistently yields slightly better performance than others. Considering the additional computational overhead of calculating node degrees, random selection may serve as a more-efficient and practical alternative.

Next, we examine how the performance of HYPERGC varies with different ranges of candidate nodes. To this end, we define the candidate node set by specifying the min/max *hop distances* to the target node that only the nodes within this range are considered as candidates. We then compute attention scores only for the candidate nodes and select the one with the highest score. The subsequent steps follow the standard procedure of HYPERGC. We set both the min/max *hop distances* in the range of 1 to 10, under the constraint that the min hop is not greater than the max hop, and evaluate all possible min-max configurations. Figure 3 presents the results on (a) Citeseer and (b) Cora, where each cell represents the accuracy difference between HYPERGC (under the given hyperparameter setting) and the second-best method. We note that the results from the other datasets show a tendency similar to those from Citeseer and Cora. We observe that HYPERGC consistently outperforms the second-best method across all combinations of the min-max hops. Notably, we observe the improved accuracy even under low min-max hop settings, indicating that our attention-based candidate node selection strategy facilitates effective representation learning while significantly reducing the computational overhead.

Finally, we investigate the impact of the number of generated hyperedges on HYPERGC’s performance. Specifically, we vary the

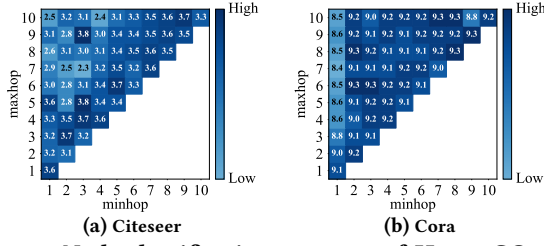


Figure 3: Node classification accuracy of HYPERGC under different min-max hop distances.

generation ratio from 10% to 100% of the total number of real hyperedges, in step of 10%. For each ratio, we evaluate the downstream task performance of HYPERGC to assess how the volume of generated hyperedges affects representation quality. Due to space constraints, we present only textual results, while the corresponding experimental figures are provided in Figure 8 of Appendix A.4. We observe that HYPERGC achieves high accuracy even with a low ratio of generated hyperedges. Moreover, HYPERGC consistently outperforms the best competing method, HypeBoy, across all generation ratios. These results suggest that HYPERGC is robust to the proportion of generated hyperedges, indicating its *insensitivity* to this hyperparameter.

EQ4: Parameter sensitivity. To investigate the *learning stability* of HYPERGC, we analyze how its accuracy varies with different combinations of loss function weights. Following [25], the weights ω_{ce} and ω_{cl} are selected from the set $[2^{-4}, 2^{-3}, \dots, 2^4]$, respectively. Figure 4 shows the results on (a) Citeseer and (b) Cora, where each cell represents the accuracy difference between HYPERGC (under the given hyperparameter setting) and the second-best method. We summarize key observations as follows. First, HYPERGC consistently outperforms the second-best method across most (*i.e.*, 70% in Citeseer and 78% in Cora) combinations of the loss weights. Second, the performance gap remains relatively stable regardless of the specific values of ω_{ce} and ω_{cl} , indicating that HYPERGC tends to be *insensitive* to the choice of loss weights.

We further analyze the sensitivity of HYPERGC to the number of LSTM layers (l) by evaluating five configurations ranging from 1 to 16. Due to space constraints, only textual results are presented here, while the corresponding figures are provided in Figure 9 of Appendix A.4. The results show that HYPERGC remains stable across different values of l , indicating low sensitivity to LSTM depth.

EQ5: Efficiency and scalability. HYPERGC may require high computational complexity due to its careful hyperedge generation and evaluation processes. To assess its overhead, we compare the training efficiency of HYPERGC with those of other competing methods. To this end, we measure the average training time per epoch for each dataset. Figure 5 shows the results in (a) Citeseer and (b) Cora. We observe that, while HYPERGC is not the fastest method, it is also not the slowest; its training time remains acceptable given that it consistently and substantially outperforms all competing methods in terms of accuracy. Moreover, reducing the number of LSTM cells (denoted as HYPERGC₅₀) yields an approximately 50% reduction in training time compared to the full setting (denoted as HYPERGC₁₀₀), substantially narrowing the efficiency gap with existing methods.

Finally, we evaluate the scalability of HYPERGC in training with the increasing size of hypergraphs. To this end, we train HYPERGC

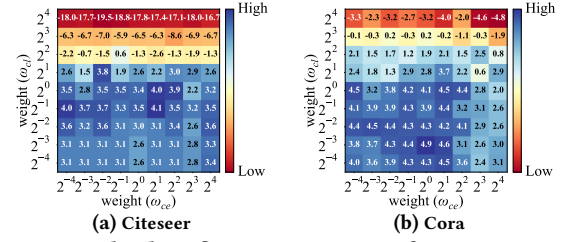


Figure 4: Node classification accuracy of HYPERGC under different loss weights.

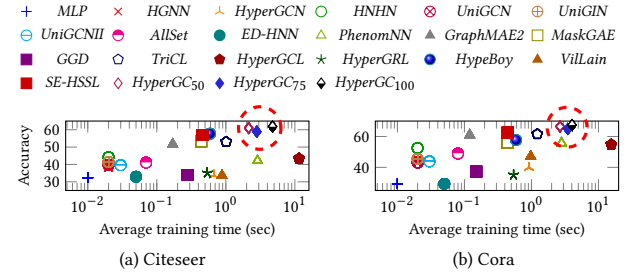


Figure 5: The accuracy of HRL methods over the average training time per epoch (sec).

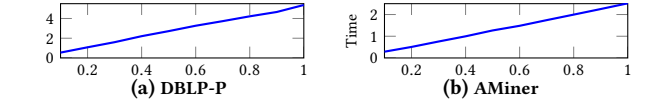


Figure 6: Training time per epoch of HYPERGC as the number of hyperedges increases.

for 20 training epochs in each dataset with varying ratios of training examples (*i.e.*, hyperedges) from 10% to 100% in increment of 10%, and measure the average training time per epoch. Figure 6 shows the results on the two largest datasets, (a) DBLP-P and (b) AMiner. We observe that HYPERGC provides almost *linear scalability* in training with the increasing size of hypergraphs.

6 Conclusions

In this paper, we present HYPERGC, a hybrid self-supervised HRL framework that combines (S1) full hyperedge reconstruction via progressive node selection enhanced with dynamic target cross-entropy (DTCE) and (S2) contrastive evaluation that removes the need for handcrafted negative samples. Extensive experiments on 11 real-world hypergraph datasets across three downstream tasks show that each strategy helps improve performance and that their combination consistently achieves the best result in all cases, outperforming 19 state-of-the-art HRL methods. Our progressive hyperedge generation—expanding from a single seed node—demonstrates strong potential for modeling dynamic and evolving environments; as future work, we plan to extend HYPERGC with more adaptive generation mechanisms for temporal hypergraphs.

Acknowledgments

The work of David Y. Kang was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (RS-2026-25494906). The work of Sang-Wook Kim was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No.RS-2022-00155586 and No.RS-2020-II201373).

References

- [1] J. Benson, R. Abebe, M. Schaub, A. Jadbabaie, and J. Kleinberg. 2018. Simplicial Closure and Higher-Order Link Prediction. *Proc. National Academy of Sciences USA* (2018).
- [2] A. Ceria and H. Wang. 2023. Temporal-topological properties of higher-order evolving networks. *Scientific Reports* (2023).
- [3] E. Chien, C. Pan, J. Peng, and O. Milenkovic. 2021. You are Allset: A Multiset Function Framework for Hypergraph Neural Networks. In *Proc. of Int'l Conf. on Learning Representations (ICLR)*.
- [4] E. Chien, C. Pan, J. Peng, and O. Milenkovic. 2021. You are Allset: A Multiset Function Framework for Hypergraph Neural Networks. In *Proc. of Int'l Conf. on Learning Representations (ICLR)*.
- [5] P. Chodrow, N. Veldt, and A. Benson. 2021. Generative Hypergraph Clustering: From Blockmodels to Modularity. *Science Advances* (2021).
- [6] L. Danon, A. Diaz-Guolera, J. Duch, and A. Arenas. 2005. Comparing Community Structure Identification. *J. Stat. Mech.* (2005).
- [7] M. Do, S. Yoon, B. Hooi, and K. Shin. 2020. Structural Patterns and Generative Models of Real-world Hypergraphs. In *Proc. of ACM Int'l Conf. on Knowledge Discovery and Data Mining (KDD)*.
- [8] Y. Dong, W. Sawin, and Y. Bengio. 2020. HNNH: Hypergraph Networks with Hyperedge Neurons. In *ICML Graph Representation Learning and Beyond Workshop*.
- [9] B. Du, C. Yuan, R. Barton, T. Neiman, and H. Tong. 2022. Self-Supervised Hypergraph Representation Learning. In *Proc. of IEEE Int'l Conf. on Big Data (Big Data)*.
- [10] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao. 2019. Hypergraph Neural Networks. In *Proc. of AAAI Conf. on Artificial Intelligence (AAAI)*.
- [11] D. Gailhard, E. Tartaglione, L. Naviner, and J. H. Giraldo. 2025. HYGENE: A Diffusion-Based Hypergraph Generation Method. In *Proc. of the AAAI Conf. on Artificial Intelligence (AAAI)*.
- [12] M. Gardner and S. Dorling. 1998. Artificial Neural Networks (the Multilayer Perceptron)—A Review of Applications in the Atmospheric Sciences. *Atmospheric environment* (1998).
- [13] Z. Han, X. Zheng, C. Chen, W. Cheng, and Y. Yao. 2023. Intra and Inter Domain HyperGraph Convolutional Network for Cross-Domain Recommendation. In *Proc. of ACM Web Conf. (WWW)*.
- [14] K. He, X. Zhang, S. Ren, and J. Sun. 2015. Heterogeneous Graph Neural Network. In *Proc. of IEEE Int'l Conf. on Computer Vision (ICCV)*.
- [15] H. Hochreiter and J. Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* (1997).
- [16] Z. Hou, Y. He, Y. Cen, X. Liu, Y. Dong, E. Kharlamov, and J. Tang. 2023. Graphmae2: A Decoding-Enhanced Masked Self-Supervised Graph Learner. In *Proc. of ACM Web Conf. (WWW)*.
- [17] J. Huang and J. Yang. 2021. UniGNN: a Unified Framework for Graph and Hypergraph Neural Networks. In *Proc. of Int'l Joint Conf. on Artificial Intelligence (IJCAI)*.
- [18] H. Hwang, S. Lee, C. Park, and K. Shin. 2022. AHP: Learning to Negative Sample for Hyperedge Prediction. In *Proc. of Int'l ACM Conf. on Research and Development in Information Retrieval (SIGIR)*.
- [19] A. Jain and R. Dubes. 1988. *Algorithms for Clustering Data*. Prentice-Hall.
- [20] D. Kang, Q. Mei, and S. Kim. 2024. Low Mileage, High Fidelity: Evaluating Hypergraph Expansion Methods by Quantifying the Information Loss. In *Proc. of ACM Web Conf. (WWW)*.
- [21] M. Kelly, R. Longjohn, and K. Nottingham. 2017. UCI Machine Learning Repository. <https://archive.ics.uci.edu>
- [22] S. Kim, S. Kang, F. Bu, S. Lee, J. Yoo, and K. Shin. 2024. HypeBoy: Generative Self-Supervised Representation Learning on Hypergraphs. In *Proc. of Int'l Conf. on Learning Representations (ICLR)*.
- [23] T. Kipf and M. Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *Proc. of Int'l Conf. on Learning Representations (ICLR)*.
- [24] T. Kobayashi, T. Takaguchi, and A. Barrat. 2019. The Structured Backbone of Temporal Social Ties. *Nature Communications* (2019).
- [25] D. Lee and K. Shin. 2023. I'm me, we're us, and i'm us: Tri-directional contrastive learning on hypergraphs. In *Proc. of AAAI Conf. on Artificial Intelligence (AAAI)*.
- [26] G. Lee, S. Y. Lee, and K. Shin. 2024. Villain: Self-Supervised Learning on Homogeneous Hypergraphs without Features via Virtual Label Propagation. In *Proc. of Int'l Conf. on World Wide Web (WWW)*.
- [27] F. Li, X. Wang, D. Cheng, W. Zhang, Y. Zhang, and X. Lin. 2024. Hypergraph Self-supervised Learning with Sampling-efficient Signals. In *Proc. of Int'l Joint Conf. on Artificial Intelligence (IJCAI)*.
- [28] J. Li, R. Wu, W. Sun, L. Chen, S. Tian, L. Zhu, C. Meng, Z. Zheng, and W. Wang. 2023. What's Behind the Mask: Understanding Masked Graph Modeling for Graph Autoencoders. In *Proc. of ACM Int'l Conf. on Knowledge Discovery and Data Mining (KDD)*.
- [29] Z. Li, Y. Lin, Y. Wang, W. Liu, M. Yu, Z. Yang, and G. Lyu. 2025. Critical Node-aware Augmentation for Hypergraph Contrastive Learning. In *Proc. of Int'l Joint Conf. on Artificial Intelligence (IJCAI)*.
- [30] M. Pan, F. Li, X. Wang, W. Zhang, and X. Lin. 2025. HiTeC: Hierarchical Contrastive Learning on Text-Attributed Hypergraph with Semantic-Aware Augmentation. *arXiv preprint arXiv:2508.03104* (2025).
- [31] P. Patil, G. Sharma, and M. Murty. 2020. Negative Sampling for Hyperlink Prediction in Networks. In *Proc. of Pacific-Asia Conf. on Knowledge Discovery and Data Mining (PAKDD)*.
- [32] L. Sun, S. Ji, and J. Ye. 2008. Spectral Learning for Multi-Label Classification. In *Proc. of ACM Int'l Conf. on Knowledge Discovery and Data Mining (KDD)*.
- [33] N. Veldt. 2023. Cut-Matching Games for Generalized Hypergraph Ratio Cuts. In *Proc. of ACM Web Conf. (WWW)*.
- [34] N. Veldt, A. Benson, and J. Kleinberg. 2020. Minimizing Localized Ratio Cut Objectives in Hypergraphs. In *Proc. of ACM Int'l Conf. on Knowledge Discovery and Data Mining (KDD)*.
- [35] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio. 2017. Graph Attention Networks. *arXiv preprint arXiv:1710.10903* (2017).
- [36] P. Wang, S. Yang, Y. Liu, Z. Wang, and P. Li. 2023. Equivariant Hypergraph Diffusion Neural Operators. In *Proc. of Int'l Conf. on Learning Representations (ICLR)*.
- [37] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, and P. Yu. 2019. Heterogeneous Graph Attention Network. In *Proc. of ACM Web Conf. (WWW)*.
- [38] Y. Wang, Q. Gan, X. Qiu, X. Huang, and D. Wipf. 2023. From Hypergraph Energy Functions to Hypergraph Neural Networks. In *Proc. of Int'l Conf. on Machine Learning (ICML)*.
- [39] T. wei, Y. You, T. Chen, Y. Shen, J. He, and Z. Wang. 2022. Augmentations in Hypergraph Contrastive Learning: Fabricated and Generative. In *Proc. of the Conf. on Advances in Neural Information Processing Systems (NeurIPS)*.
- [40] W. Wen and T. Yu. 2025. HyperPLR: Hypergraph Generation through Projection, Learning, and Reconstruction. In *Proc. of Int'l Conf. on Learning Representations (ICLR)*.
- [41] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao. 2015. 3D ShapeNets: A Deep Representation for Volumetric Shapes. In *Proc. of IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*.
- [42] N. Yadti, M. Nimishakavi, P. Yadav, V. Nitin, A. Louis, and P. Talukdar. 2019. HyperGCN: A New Method of Training Graph Convolution Networks on Hypergraphs. In *Proc. of Conf. on Neural Information Processing Systems (NeurIPS)*.
- [43] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen. 2020. Graph Contrastive Learning with Augmentations. In *Proc. of the Conf. on Advances in Neural Information Processing Systems (NeurIPS)*.
- [44] J. Yu, H. Yin, J. Li, Q. Wang, N. Hung, and X. Zhang. 2021. Self-Supervised Multi-Channel Hypergraph Convolutional Network for Social Recommendation. In *Proc. of ACM Web Conf. (WWW)*.
- [45] C. Zhang, D. Song, C. Huang, A. Swami, and N. Chawla. 2019. Heterogeneous Graph Neural Network. In *Proc. of ACM Int'l Conf. on Knowledge Discovery and Data Mining (KDD)*.
- [46] R. Zhang, Y. Zou, and J. Ma. 2020. Hyper-SAGNN: A Self-Attention Based Graph Neural Network for Hypergraphs. In *Proc. of Int'l Conf. on Learning Representations (ICLR)*.
- [47] Y. Zheng, S. Pan, V. Lee, Y. Zheng, and P. Yu. 2022. Rethinking and Scaling Up Graph Contrastive Learning: An Extremely Efficient Approach With Group Discrimination. In *Proc. of the Conf. on Advances in Neural Information Processing Systems (NeurIPS)*.
- [48] D. Zhou, J. Huang, and B. Scholköpfung. 2006. Learning with hypergraphs: Clustering, classification, and embedding. In *Proc. of Conf. on Neural Information Processing Systems (NeurIPS)*.

A Appendix

A.1 Analysis of Real-World Hyperedge Formation Mechanisms

In this section, we present additional experimental results that support the observations in Section 3 regarding real-world hyperedge formation mechanisms. The results are summarized in Table 8. We observe that, across most real-world hypergraphs, the vast majority of hyperedges (on average, over 99%) are formed through a progressive expansion process. In addition, a small portion of hyperedges exhibit alternative formation patterns, such as node removal or the emergence of entirely new groupwise relationships. These findings suggest that our progressive expansion strategy aligns well with real-world hyperedge formation dynamics and contributes to the improved performance of HRL methods.

Table 8: Proportion of hyperedge formation mechanisms on four real-world temporal hypergraphs

Dataset	Expansion	Contraction	New Structure
email-Eu	99.67	0.07	0.25
email-Enron	99.12	0.15	0.71
tags-math-sx	99.97	0.01	0.02
threads-math-sx	97.82	0.01	2.17
SFHH-conference	95.86	3.32	0.82
science-gallery	99.10	0.67	0.23
malawi-village	100	0.00	0.00
InVS15	100	0.00	0.00
hypertext-conference	99.58	0.42	0.00
hospital-lyon	100	0.00	0.00

A.2 Impact of Node Ordering in Hyperedge Generation on Node Classification Accuracy of HYPERGC

As discussed in the main text, hyperedges represent groupwise relationships among multiple nodes and are therefore naturally interpreted as unordered sets, whose semantics are invariant to node ordering. Although HYPERGC generates hyperedges progressively using a recurrent model—where nodes are selected sequentially and an implicit ordering may arise—this ordering is solely a modeling mechanism to capture contextual information from previously selected nodes and is not intended to alter hyperedge semantics. Consequently, the final generated hyperedge remains permutation-invariant. In this section, we empirically examine whether different generation orders affect downstream performance. To this end, we run HYPERGC ten times on the node classification task, fixing the final hyperedge compositions while varying the node selection order during hyperedge generation. We then compute the standard deviation of node classification accuracy across runs. As shown in Table 9, the standard deviation remains consistently low, indicating that variations in node selection order have little impact on downstream task performance.

Table 9: Standard deviation of node classification accuracy across 10 random permutations

Dataset	Citeseer	Cora	DBLP-A	Cora-CA	IMDB	House
STD	0.17	0.08	0.91	0.53	0.40	0.44

Dataset	Pubmed	DBLP-P	Aminer	MN-40	20News	Average
STD	0.72	0.05	0.00	0.01	0.02	0.30

A.3 Experimental Environments

Reproducibility. The experiments are conducted on the Linux server equipped with an AMD 7960X, 2TB NVMe SSD, 128GB DDR5 memory, and GeForce RTX A6000(48G). For HRL methods, we use the source codes provided by the authors of the original papers [3, 9, 10, 16, 17, 22, 25, 28, 36, 38, 39, 42, 47]. For parameters for each method, we use the best setting found via extensive grid search in the ranges suggested in its respective paper. We carefully tune the hyperparameters of our HYPERGC via grid search over the following ranges. The loss weights ω_n , ω_h , ω_{ce} and ω_{cl} are selected from $[2^{-4}, 2^{-3}, \dots, 2^4]$. The temperature parameters for contrastive loss, τ_n and τ_h , are chosen from $[0.1, 0.2, \dots, 1.0]$. For

hypergraph augmentation, the feature, incidence, and hyperedge masking rates are all varied within $[0.1, 0.2, \dots, 1.0]$. The hidden dimension is selected from $[2^6, 2^7, \dots, 2^{10}]$, and the learning rate is from $[10^{-5}, 5 \times 10^{-5}, 10^{-4}, 5 \times 10^{-4}, 10^{-3}]$. The weight decay is tuned over $[10^{-5}, 5 \times 10^{-5}, 10^{-4}, 5 \times 10^{-4}]$. The numbers of encoder layers and LSTM layers are selected from $[2^0, 2^1, \dots, 2^4]$.

A.4 Additional Experimental Results

In this section, we present additional experimental results for three downstream tasks: node classification (NC), hyperedge prediction (HP), and community detection (CD).

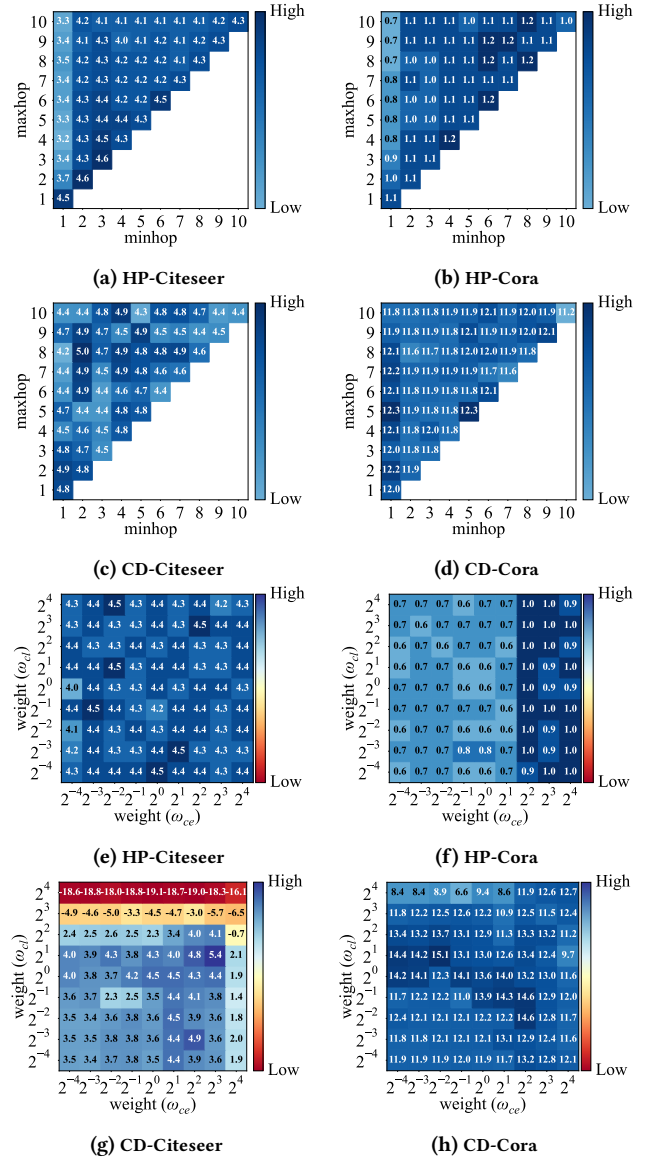
**Figure 7: Downstream task accuracy of HYPERGC under different min-max hop distances (: a-d) and loss weights (EQ4: e-h).**

Table 10: Ablation study: Accuracy comparison of five variants of HYPERGC on HP (upper) and CD (lower) (EQ1).

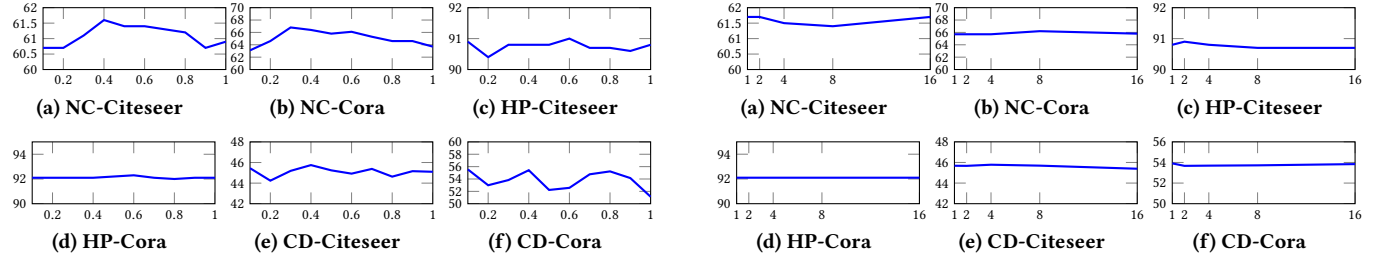
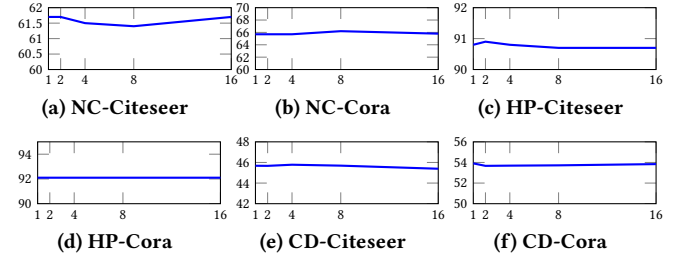
	FR	CL	Citeseer	Cora	DBLP-A	Cora-CA	IMDB	House	Pubmed	DBLP-P	Aminer	MN-40	20News	AR
V1	✗	✗	73.5 ± 3.5	83.9 ± 2.2	81.9 ± 1.2	68.5 ± 3.4	48.8 ± 2.5	86.0 ± 2.9	90.8 ± 1.1	65.0 ± 15.2	85.4 ± 1.4	99.8 ± 0.0	85.9 ± 6.9	4.2
V2	✗	✓	89.7 ± 1.4	92.1 ± 1.3	90.8 ± 1.0	83.9 ± 1.5	63.2 ± 2.8	86.6 ± 3.2	95.8 ± 0.3	50.5 ± 2.2	56.4 ± 9.8	99.8 ± 0.0	96.4 ± 2.7	2.8
V3 (w/o DTCE)	✓	✗	85.2 ± 1.4	90.8 ± 1.4	89.2 ± 1.4	85.2 ± 1.7	63.4 ± 2.4	85.3 ± 3.0	95.1 ± 0.5	94.3 ± 1.3	79.9 ± 1.2	99.8 ± 0.0	94.8 ± 3.7	3.1
V4 (w/o DTCE)	✓	✓	85.5 ± 1.7	91.0 ± 1.4	89.3 ± 1.5	84.9 ± 2.8	63.4 ± 2.4	85.7 ± 2.9	94.9 ± 0.8	93.7 ± 1.4	79.9 ± 1.2	99.8 ± 0.0	95.6 ± 3.9	3.0
V5 (w/ DTCE)	✓	✓	90.8 ± 1.2	92.8 ± 1.2	91.0 ± 1.0	89.3 ± 1.1	63.6 ± 2.3	88.0 ± 3.0	95.8 ± 0.4	96.3 ± 0.1	89.3 ± 1.4	99.9 ± 0.0	97.5 ± 2.2	1.1
V1	✗	✗	21.65	23.46	17.41	20.92	10.10	11.41	33.58	55.09	30.00	95.29	40.01	3.3
V2	✓	✓	40.21	52.25	48.40	15.87	11.09	4.92	30.43	52.80	29.75	94.73	39.77	3.9
V3 (w/o DTCE)	✗	✗	45.12	55.48	48.44	20.74	0.48	11.77	32.72	39.57	31.02	94.03	32.81	3.4
V4 (w/o DTCE)	✓	✓	45.01	52.13	47.18	47.68	8.69	11.61	32.95	62.12	40.36	95.10	39.02	2.7
V6 (w/ DTCE)	✓	✓	44.93	55.8	52.35	42.83	11.5	11.57	35.33	66.25	41.32	94.82	39.8	1.7

Table 11: HP (upper) and CD (lower) accuracy of HYPERGC according to recurrent model (EQ3)

	Citeseer	Cora	DBLP-A	Cora-CA	IMDB	House	Pubmed	DBLP-P	Aminer	MN-40	20News	AR
RNN	90.7 ± 1.5	91.9 ± 1.1	90.7 ± 1.0	89.2 ± 1.4	63.4 ± 3.1	88.4 ± 2.9	95.6 ± 0.4	96.3 ± 0.1	87.6 ± 1.7	99.9 ± 0.0	95.8 ± 3.6	2.3
GRU	90.9 ± 1.4	92.0 ± 1.2	90.5 ± 1.0	89.0 ± 1.7	63.4 ± 2.5	88.2 ± 3.2	95.8 ± 0.3	96.3 ± 0.1	88.5 ± 1.1	99.9 ± 0.0	97.4 ± 2.0	1.8
LSTM	90.8 ± 1.2	92.8 ± 1.2	91.0 ± 1.0	89.3 ± 1.1	63.6 ± 2.3	88.0 ± 3.0	95.8 ± 0.4	96.3 ± 0.1	89.3 ± 1.4	99.9 ± 0.0	97.5 ± 2.2	1.4
RNN	45.31	54.83	51.22	44.37	10.89	10.94	36.59	62.47	41.29	95.43	39.97	2.0
GRU	42.86	55.06	53.57	45.23	11.48	6.49	31.99	64.53	41.07	94.89	39.61	2.3
LSTM	44.93	55.8	52.35	42.83	11.5	11.57	35.33	66.25	41.32	94.82	39.8	1.7

Table 12: HP (upper) and CD (lower) accuracy of HYPERGC according to seed node selection (EQ3)

	Citeseer	Cora	DBLP-A	Cora-CA	IMDB	House	Pubmed	DBLP-P	Aminer	MN-40	20News	AR
High degree	90.9 ± 1.5	92.1 ± 1.2	90.5 ± 1.0	90.0 ± 0.9	63.3 ± 2.4	88.0 ± 2.6	95.7 ± 0.2	96.2 ± 0.2	88.4 ± 0.8	99.9 ± 0.0	97.6 ± 2.2	2.0
Low degree	90.6 ± 1.6	92.1 ± 1.2	90.6 ± 0.8	89.8 ± 1.1	63.4 ± 2.2	88.0 ± 2.2	95.7 ± 0.2	96.2 ± 0.1	88.2 ± 0.9	99.9 ± 0.0	97.7 ± 2.5	1.9
Random	90.8 ± 1.2	92.8 ± 1.2	91.0 ± 1.0	89.3 ± 1.1	63.6 ± 2.3	88.0 ± 3.0	95.8 ± 0.4	96.3 ± 0.1	89.3 ± 1.4	99.9 ± 0.0	97.5 ± 2.2	1.6
High degree	45.35	53.40	48.16	43.05	11.26	11.57	32.88	58.37	41.26	95.14	38.04	2.1
Low degree	45.68	53.23	48.31	43.14	11.19	6.72	32.22	58.79	41.35	94.98	38.03	2.2
Random	44.93	55.8	52.35	42.83	11.5	11.57	35.33	66.25	41.32	94.82	39.8	1.6

**Figure 8: Downstream task accuracy of HYPERGC with increasing proportion of generated hyperedges (EQ3).****Figure 9: Downstream task accuracy of HYPERGC under different numbers of LSTM layers (EQ4).**